

Core Principles of Greedy Algorithms



Hot Topic: Is the most intuitive choice always the best one in computing? No, that's why we use Dynamic Programming

How to join server?

How to take roles?

How to join group on CF?

How to make contest like these?

What is Greedy Thinking?

Current best choice

Greedy thinking is essentially "living in the moment." It looks at the current options and picks the absolute best one available *right now*, without worrying about future consequences.

Greedy thinking assumes that a series of local optimizations will naturally lead to a global optimum.

It works when the problem structure ensures that current "greedy" gains never block superior "future" gains.

$O(n)$

Typical Time Complexity

Greedy algorithms are often the fastest possible approach. While Dynamic Programming might take $O(n^2)$



Basic Example: Making Change

The Goal: Give a customer change using the fewest coins possible.

Denomination of coins = [1,2,5,10,20,50,100,200,500,1000]

Suppose we want to give 853\$

What should be the answer?



| Basic Example: Making Change

Answer: [500,200,100,50,2,1]



The Choice: Always pick the largest denomination that is less than or equal to the remaining amount.



Why it works: Because our currencies are designed to follow Greedy Approach (For Laymans)

Try This: If denominations are {1, 3, 4} and we need 6\$?

Answer via above approach: [4,1,1]

Correct Answer: [3,3]

Can you explain this?

Canonical Coin System

In the "Making Change" problem, greedy thinking works because the set of available coins follows a specific mathematical property (Matroids).

[1 2 5 10 20 50 100 200]

[= 1]

[= 3]

[= 8]

[= 18]

[= 38]

[= 88]

153

"So, we will choose 100 here, and if we don't choose 100 then it will cost us more coins (at least two extra coins)."

Basic Example: Making Change

Implementation

```
cpp

vector<int> coins = {1, 2, 5, 10, 20, 50, 100, 200};

int canonicalCoinChange(int amount) {
    int cnt = 0;
    for (int i = coins.size() - 1; i >= 0; i--) {
        cnt += amount / coins[i];
        amount %= coins[i];
    }
    return cnt;    // minimum number of coins (canonical system)
}
```

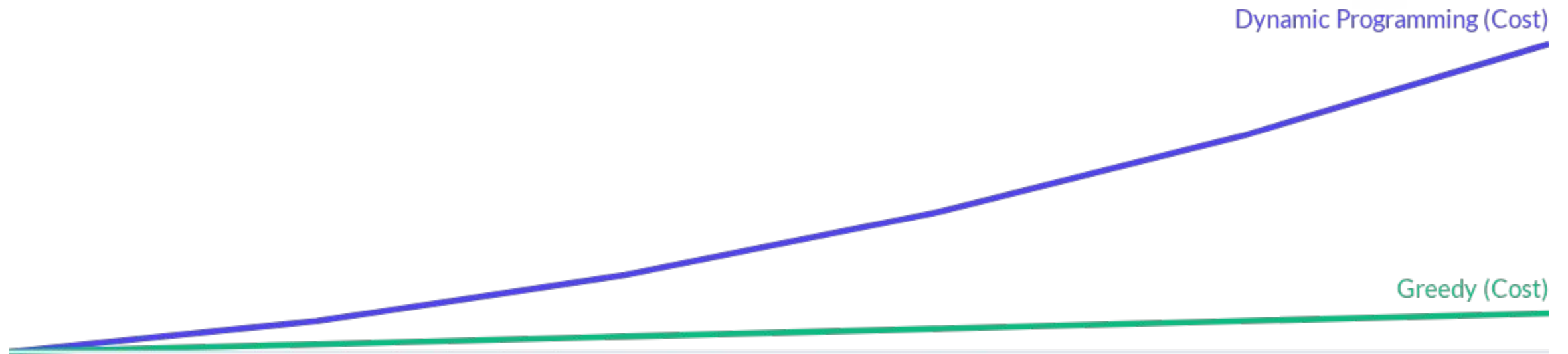
Classic Greedy Solutions

Problem Type	Algorithm	Greedy Choice	Complexity
Shortest Path	Dijkstra's	Pick closest unvisited node	$O(E \log V)$
Data Compression	Huffman Coding	Merge two lowest frequencies	$O(n \log n)$
Spanning Tree	Prim's / Kruskal's	Pick minimum weight edge	$O(E \log V)$
Resource Loading	Fractional Knapsack	Pick highest value density	$O(n \log n)$



Hot Topic: Why is Kruskal's algorithm considered "global" yet greedy?

Efficiency vs. Complexity



As input size grows, Greedy algorithms maintain a low memory and time overhead compared to exhaustive Dynamic Programming.

⚡ *Advanced: In what scenarios would you sacrifice optimality for greedy speed?*



More Applications

Moving from basic intuition to robust algorithm design

Fractional Knapsack

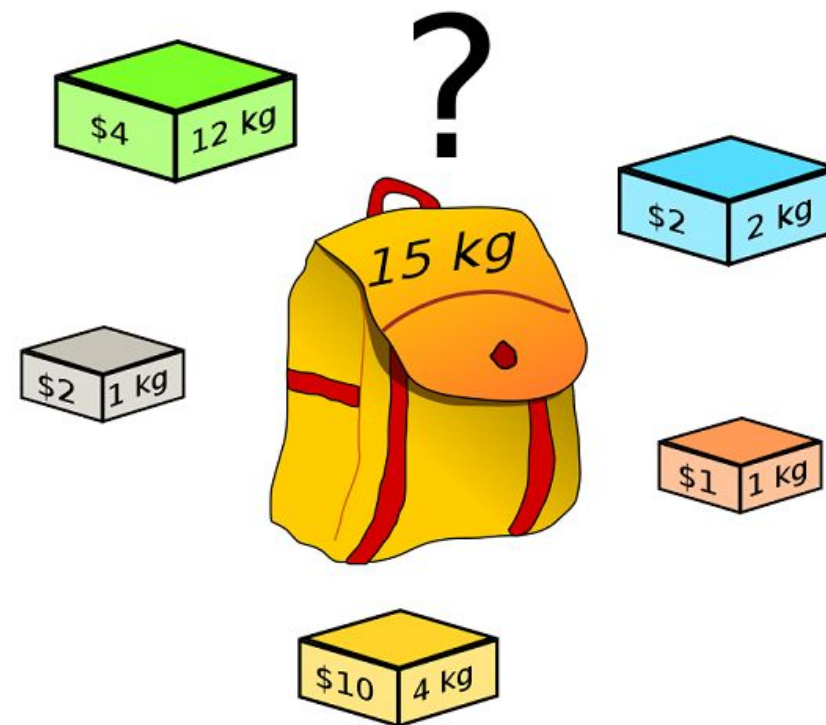
Problem Statement: Given n items, each with: (Value v_i and Weight w_i) and a knapsack of capacity W , choose items (fractions allowed) to maximize total value.

Item	Value (v)	Weight (w)	v / w
A	60	10	6.0
B	100	20	5.0
C	120	30	4.0

Greedy Choice: Sort items by value density:

$$\text{Density} = \frac{v_i}{w_i}$$

Always pick the item with the highest density first.



? CHALLENGE: If items are not divisible (0/1 Knapsack), why does this greedy strategy fail? Find a counterexample with 3 items.

0/1 Knapsack Problem

Problem Statement: Given n items, each with: (Value v_i and Weight w_i) and a knapsack of capacity W , choose items to maximize total value. Each item is either taken fully (1) or not taken (0).

Greedy Choices: Pick item with maximum value/weight ratio

Item	Value	Weight	v/w
A	60	10	6
B	100	20	5
C	120	30	4

Knapsack Capacity = 50

Greedy Value = 160 (Pick A, Pick B)

Optimal Value = 220 (Pick B, Pick C)

“All greedy strategies fail in 0/1 knapsack because local optimal choices can block globally optimal combinations.”



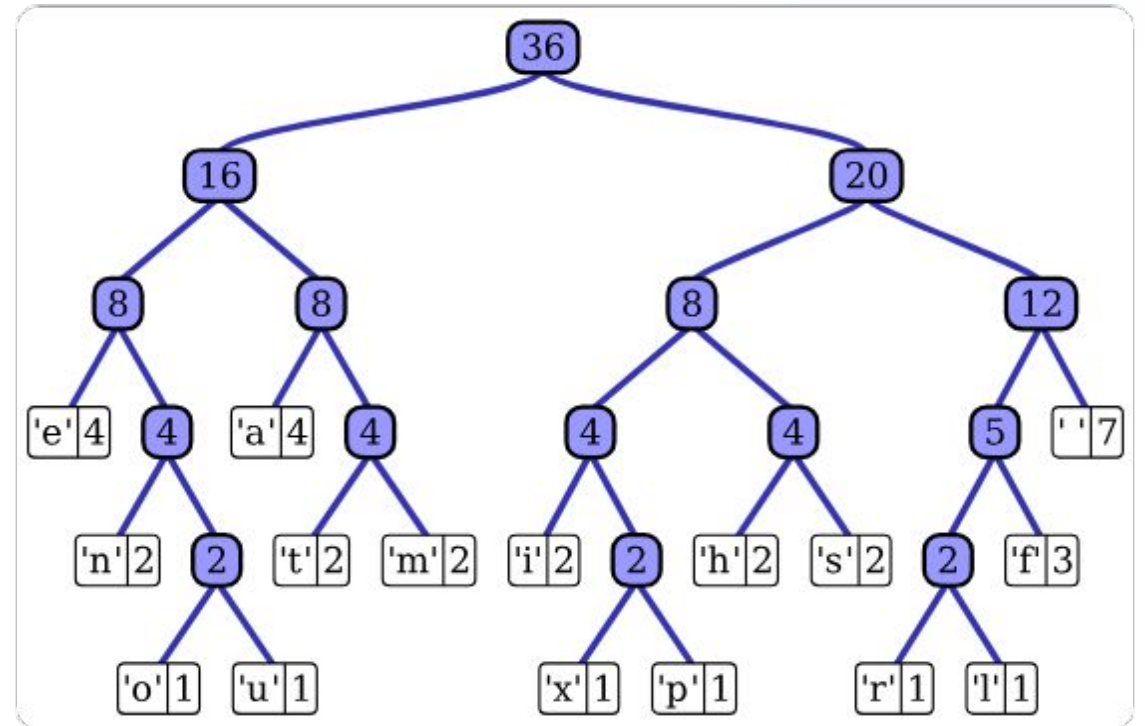
Dynamic Programming

Encoding: Huffman Trees

Used for lossless data compression. Assigns shorter binary codes to more frequent characters.

Steps:

1. Count frequency of each char.
2. Greedily merge two nodes with lowest frequencies.
3. Repeat until one tree remains.
- .



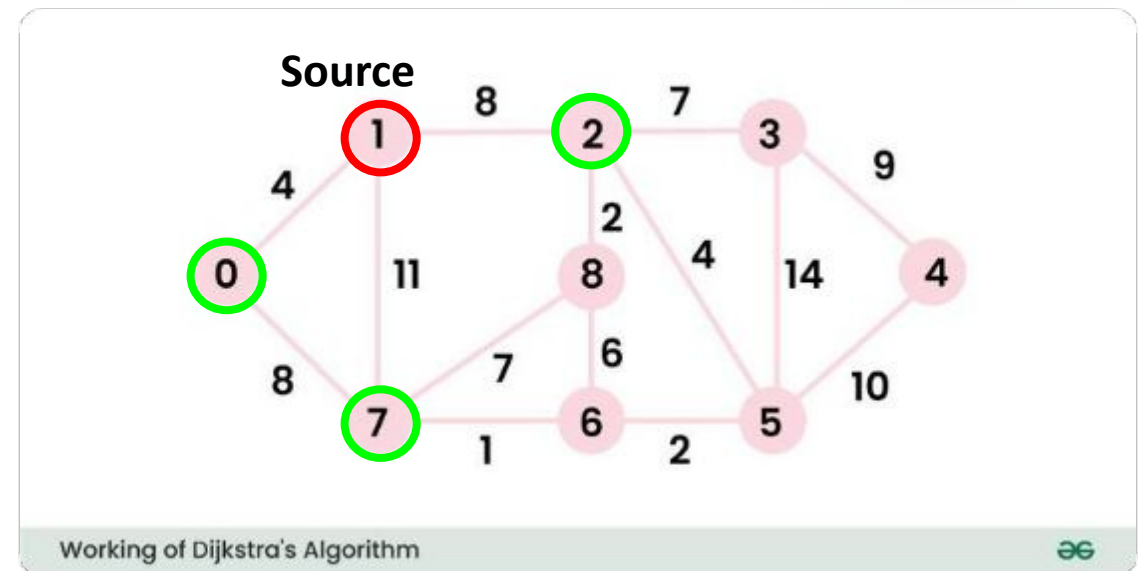
⚡ ADVANCED: What is the 'Prefix Property' in Huffman coding and why is it essential for unique decoding?

Pathfinding: Dijkstra's Algorithm

Finds the shortest path from a source to all other vertices in a **positive** weighted graph.

Greedy Choice: At each step, visit the "unvisited" vertex with the smallest known distance from the source.

Complexity: $O(E \log V)$ with Min-Priority Queue.



! LIMITATION: Why does Dijkstra's algorithm fail with negative edge weights? Explore the Bellman-Ford alternative.

Job Scheduling: The Problem

INPUT:

- A set J of n jobs $\{j_1, j_2, \dots, j_n\}$
- job j_i is specified by two real numbers
 - $s(i)$: start time of job j_i
 - $f(i)$: finish time of job j_i
- A single server

Constraints:

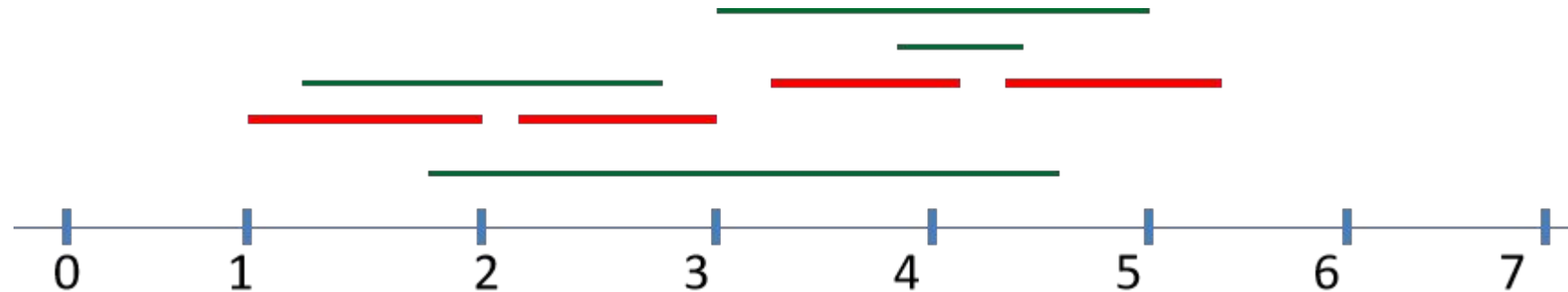
- Server can execute at most one job at any moment of time.
- Job j_i , if scheduled, has to be scheduled during $[s(i), f(i)]$ only.

Aim:

To select the **largest** subset of non-overlapping jobs which can be executed by the server.

Job Scheduling: The Problem

INPUT: $(1, 2)$, $(1.2, 2.8)$, $(1.8, 4.6)$, $(2.1, 3)$, $(3, 5)$, $(3.3, 4.2)$, $(3.9, 4.4)$, $(4.3, 5.4)$



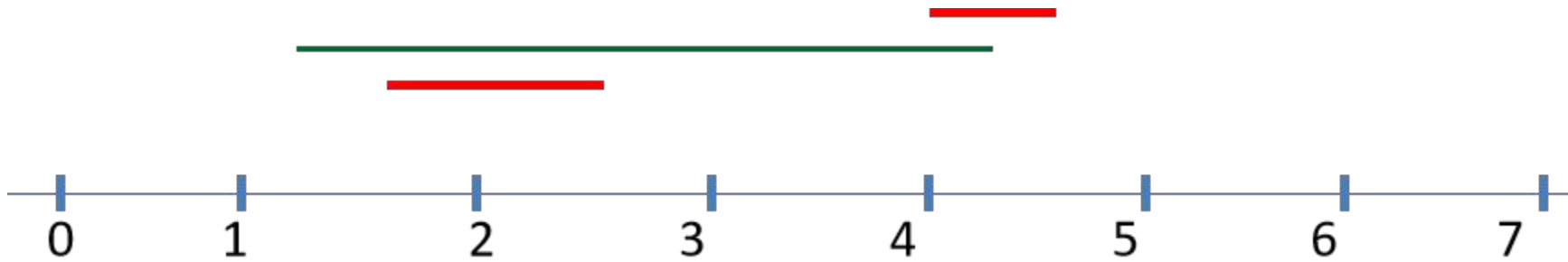
job i is said to be **non-overlapping** with job k if $(s(i), f(i)) \cap (s(k), f(k)) = \emptyset$

| Job Scheduling: The Problem

Strategy 1: Select the earliest start time job

Job Scheduling: The Problem

Strategy 1: Select the earliest start time job



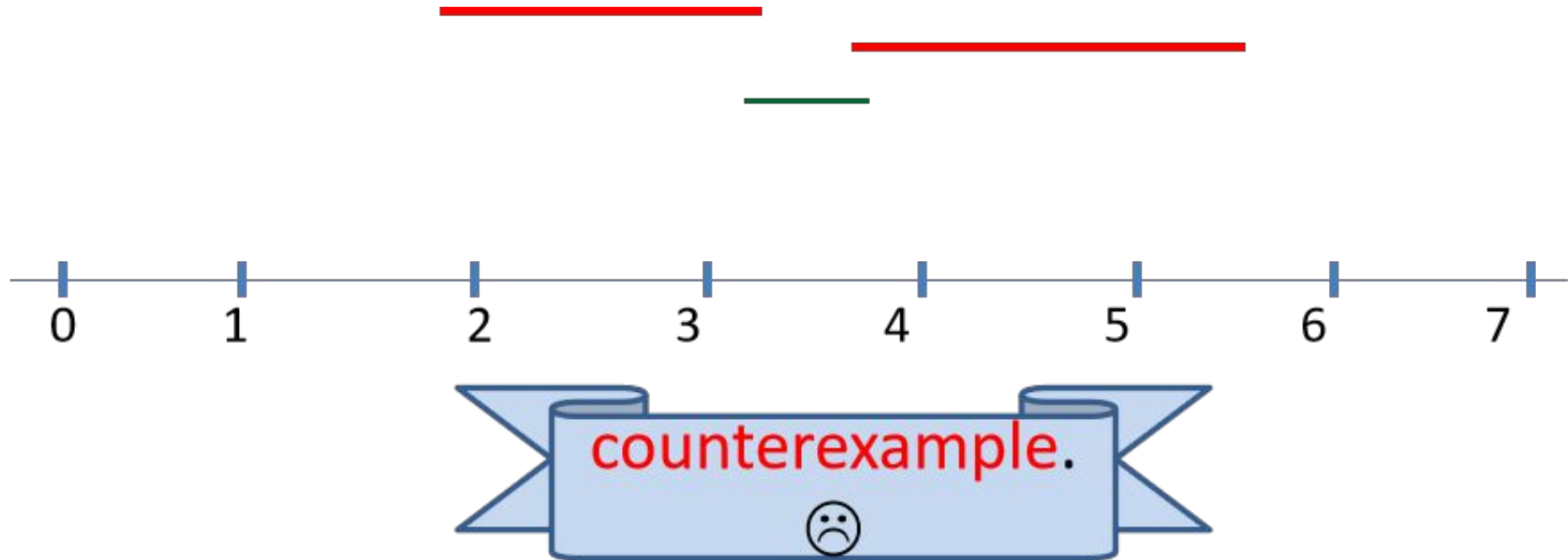
X Fails

| Job Scheduling: The Problem

Strategy 2: Select the job with smallest duration

Job Scheduling: The Problem

Strategy 2: Select the job with smallest duration



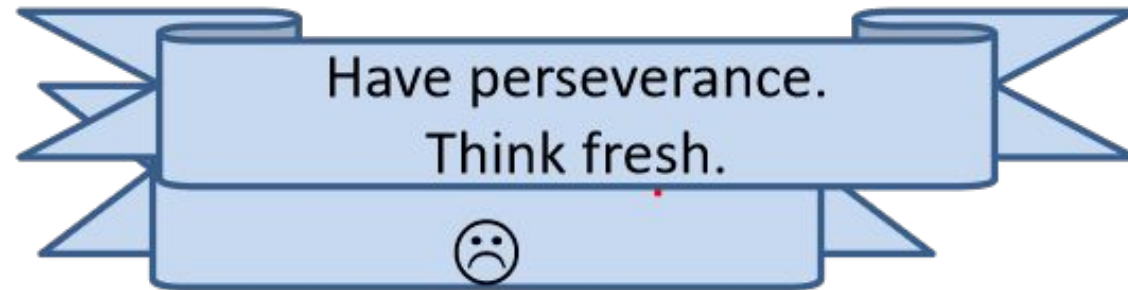
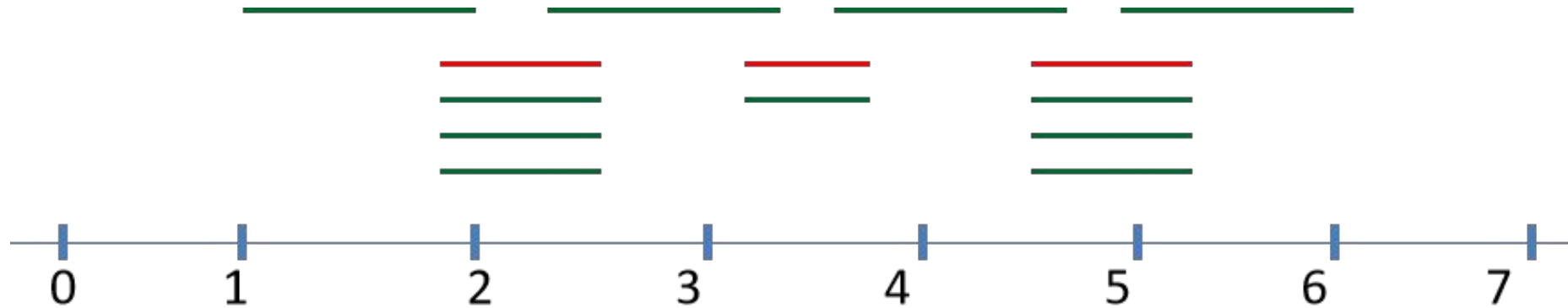
X Fails

| Job Scheduling: The Problem

Strategy 3: Select the job with smallest no. of overlaps

Job Scheduling: The Problem

Strategy 3: Select the job with smallest no. of overlaps



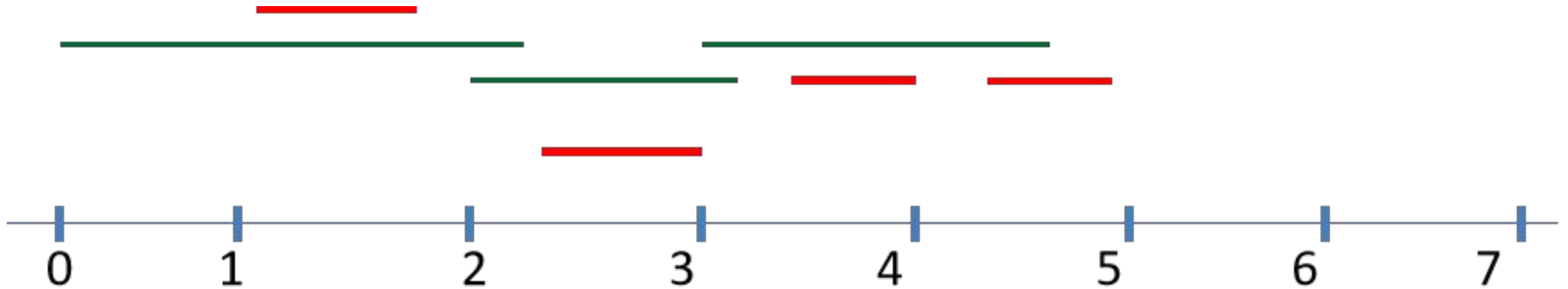
X Fails

| Job Scheduling: The Problem

Strategy 4: Select the job with earliest finish time

Job Scheduling: The Problem

Strategy 3: Select the job with earliest finish time



It is indeed a correct solution for this example. We should try to prove the correctness of the algorithm. 😊 But first we give a full description of the algorithm based on this strategy.

True Solution, Implement this approach in code with $O(N \log N)$ TC

Room Allocation: The Problem

Problem Statement: Given a list of bookings with:

Arrival time a_i & Departure time d_i – for each customer.

Goal: Allocate rooms such that:

No overlapping bookings share a room & minimum number of rooms are used

Correct Greedy Strategy

Sort jobs by **Start Time**. For each job, assign it to an available room. If no room is free, open a new room.

Proof of Correctness

If the algorithm opens room 'd', then at that moment there were 'd' jobs overlapping. Since at least 'd' rooms are required to handle 'd' overlaps, 'd' is the global minima.

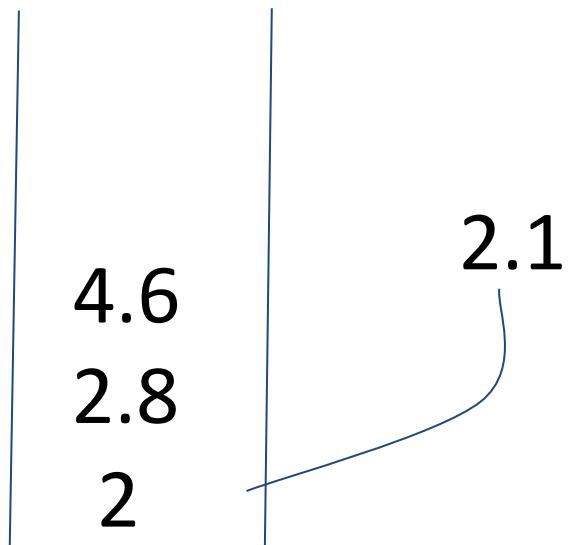
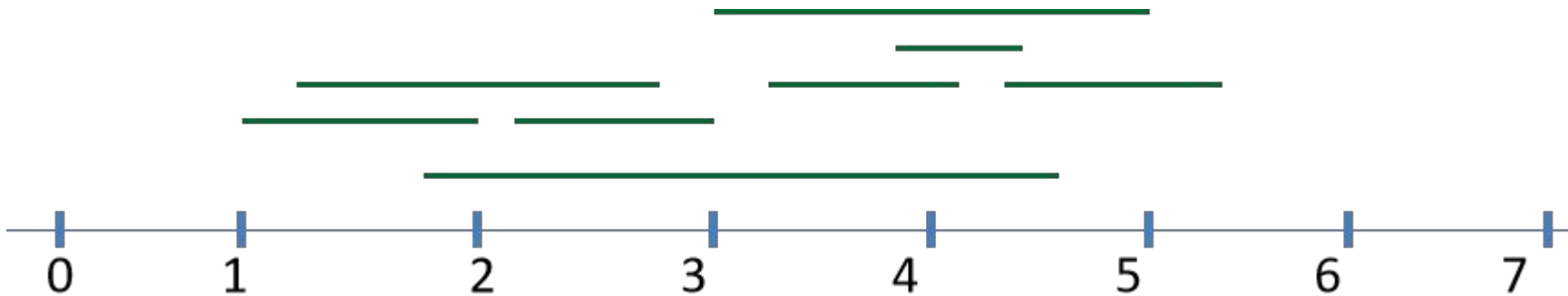
Implementation: Sort bookings by arrival time

Use a min-heap ordered by departure time

Reuse rooms that free up earliest

Room Allocation: The Problem

(a_i, d_i) – $(1, 2), (1.2, 2.8), (1.8, 4.6), (2.1, 3), (3, 5), (3.3, 4.2), (3.9, 4.4), (4.3, 5.4)$



→ 4 is the maximum number of rooms required
→ maximum size of min-heap will be the maximum rooms required

Optimal Task Scheduling

Problem Statement: You are given a set of tasks (jobs), where:

Each task takes 1 unit of time

Each task has a deadline

Each task gives a profit if completed before or on its deadline

Goal: Maximize profit by scheduling non-overlapping tasks with deadlines.

A job completed at time $t \leq d_i$ yields profit r_i , otherwise it yields 0 profit.

Task	Deadline (T)	Profit (\$)	Greedy Choice?
Task A	2	100	Yes (Highest Profit)
Task B	1	19	No
Task C	2	27	Yes

Optimal Task Scheduling

Problem Statement: You are given a set of tasks (jobs), where:

Each task takes 1 unit of time

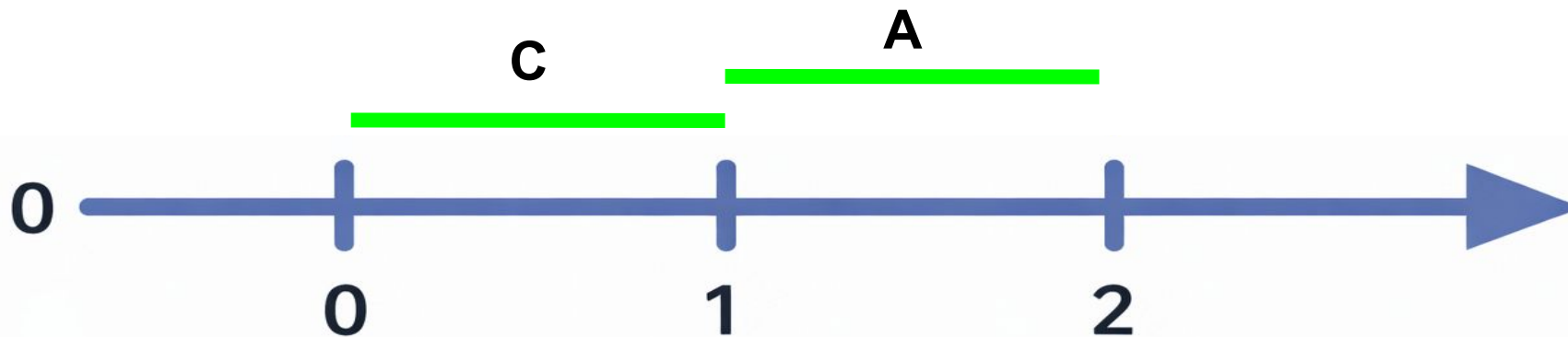
Each task has a deadline

Each task gives a profit if completed before or on its deadline

Goal: Maximize profit by scheduling non-overlapping tasks with deadlines.

A job completed at time $t \leq d_i$ yields profit r_i , otherwise it yields 0 profit.

Strategy: Sort tasks by profit and place each task as late as possible (before its deadline).



THANKYOU

Mastering Greedy isn't about the code; it's about proving the logic holds.
